

# SXiaum

---

A MEV-Resistant, Parallel-Execution  
Layer 1 Blockchain with Verkle State Cryptography

Technical Whitepaper • v1.0 • April 2026

---

**Praful V Raj**

Lead Developer & Architect

*and The VageChain Open Source Community*

---

**GitHub** [github.com/VageChain](https://github.com/VageChain)

**Twitter** [@VageChain](https://twitter.com/VageChain)

**Discord** [discord.gg/3tX6kWTzEs](https://discord.gg/3tX6kWTzEs)

**License** MIT License

---

## Table of Contents

---

### Contents

---

|          |                                                             |           |
|----------|-------------------------------------------------------------|-----------|
| <b>1</b> | <b>Abstract</b>                                             | <b>4</b>  |
| <b>2</b> | <b>Introduction &amp; Motivation</b>                        | <b>4</b>  |
| 2.1      | The Promise and Betrayal of Decentralized Finance . . . . . | 4         |
| 2.2      | Why Existing Solutions Fall Short . . . . .                 | 5         |
| 2.3      | The VageChain Thesis . . . . .                              | 5         |
| 2.4      | Key Contributions . . . . .                                 | 5         |
| <b>3</b> | <b>The MEV Crisis: A Formal Problem Statement</b>           | <b>5</b>  |
| 3.1      | Definition and Taxonomy . . . . .                           | 5         |
| 3.2      | Attack Vectors . . . . .                                    | 6         |
| 3.3      | Economic Impact . . . . .                                   | 6         |
| 3.4      | The Root Cause . . . . .                                    | 6         |
| <b>4</b> | <b>Design Philosophy &amp; Principles</b>                   | <b>6</b>  |
| <b>5</b> | <b>System Architecture Overview</b>                         | <b>7</b>  |
| 5.1      | Layered Topology . . . . .                                  | 7         |
| 5.2      | Execution-Consensus Decoupling . . . . .                    | 7         |
| 5.3      | Technical Specifications . . . . .                          | 8         |
| <b>6</b> | <b>Consensus Engine: Pipelined HotStuff BFT</b>             | <b>8</b>  |
| 6.1      | Theoretical Foundation . . . . .                            | 8         |
| 6.2      | Communication Complexity . . . . .                          | 8         |
| 6.3      | Safety and Liveness Guarantees . . . . .                    | 9         |
| 6.4      | Pipelined Protocol Phases . . . . .                         | 9         |
| 6.5      | Quorum Certificate Structure . . . . .                      | 9         |
| <b>7</b> | <b>Native MEV Protection: Commit-Reveal Cryptography</b>    | <b>10</b> |
| 7.1      | Commit-Reveal Protocol . . . . .                            | 10        |
| 7.2      | Information Asymmetry Elimination . . . . .                 | 10        |
| 7.3      | Transparent RPC Integration . . . . .                       | 10        |
| <b>8</b> | <b>Parallel EVM Execution Engine</b>                        | <b>11</b> |
| 8.1      | Optimistic Concurrency Control (OCC) . . . . .              | 11        |
| 8.2      | Access Matrix Optimization . . . . .                        | 11        |
| 8.3      | Benchmark Scaling . . . . .                                 | 11        |

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>9 Verkle Tree State Cryptography</b>                     | <b>12</b> |
| 9.1 The Bottleneck of Hexary Tries . . . . .                | 12        |
| 9.2 Vector Commitments & Polynomials . . . . .              | 12        |
| 9.3 Proof Sizes and Branching Factor . . . . .              | 12        |
| <b>10 Gas Model &amp; Economic Design</b>                   | <b>13</b> |
| 10.1 Execution Gas Schedule . . . . .                       | 13        |
| 10.2 Intrinsic Gas Calculation . . . . .                    | 13        |
| <b>11 Network Layer &amp; P2P Architecture</b>              | <b>13</b> |
| <b>12 RPC API &amp; Ethereum Compatibility Layer</b>        | <b>14</b> |
| <b>13 ZK-Ready Infrastructure</b>                           | <b>14</b> |
| <b>14 Node Architecture &amp; Operational Modes</b>         | <b>14</b> |
| 14.1 Operating Modes . . . . .                              | 14        |
| 14.2 Hardware Requirements . . . . .                        | 15        |
| <b>15 Genesis Configuration &amp; Validator Economics</b>   | <b>15</b> |
| <b>16 Block Explorer &amp; Analytics Infrastructure</b>     | <b>15</b> |
| <b>17 CLI &amp; Developer Tooling</b>                       | <b>15</b> |
| <b>18 Performance Benchmarks &amp; Competitive Analysis</b> | <b>16</b> |
| 18.1 Benchmark Targets . . . . .                            | 16        |
| 18.2 Throughput Analysis . . . . .                          | 16        |
| 18.3 Competitive Landscape . . . . .                        | 16        |
| <b>19 Security Model &amp; Threat Analysis</b>              | <b>17</b> |
| <b>20 Flexibility &amp; Future Horizons</b>                 | <b>17</b> |
| 20.1 Roadmap . . . . .                                      | 17        |
| <b>21 Governance &amp; Community</b>                        | <b>18</b> |
| <b>22 Conclusion</b>                                        | <b>18</b> |
| <b>A Gas Schedule Reference</b>                             | <b>20</b> |
| <b>B DevNet Configuration</b>                               | <b>20</b> |
| <b>C Crate Architecture</b>                                 | <b>21</b> |
| <b>D Supported RPC Methods</b>                              | <b>21</b> |
| <b>E Glossary</b>                                           | <b>22</b> |
| <b>F The Ethereum Synergy — Why EVM?</b>                    | <b>22</b> |

## 1 Abstract

---

VAGECHAIN is a next-generation Layer 1 blockchain protocol engineered from first principles to solve three fundamental problems that afflict every major smart-contract platform today: Maximal Extractable Value (MEV) exploitation, sequential execution bottlenecks, and state storage bloat. By unifying a **Commit-Reveal encrypted mempool**, a **Parallel EVM execution engine** based on Optimistic Concurrency Control (OCC), **Pipelined HotStuff BFT consensus** with deterministic sub-second finality, and **Verkle Tree state cryptography** with succinct polynomial proofs, VAGECHAIN delivers a vertically integrated architecture that is simultaneously MEV-resistant, high-throughput, stateless-ready, and fully Ethereum-compatible.

The protocol achieves **4,500+ transactions per second** on simple transfers with **1.2-second deterministic finality** and **~2.5 KB state proofs**—representing a 40× improvement in proof efficiency over legacy Merkle Patricia Tries. VAGECHAIN is implemented entirely in Rust for maximum performance and memory safety, and its state transition function is architected to be ZK-provable using SNARKs/STARKs frameworks such as SP1 and Groth16.

This whitepaper presents the theoretical foundations, engineering architecture, cryptographic primitives, economic model, and operational infrastructure of VAGECHAIN in comprehensive detail.

## 2 Introduction & Motivation

---

### 2.1 The Promise and Betrayal of Decentralized Finance

---

Blockchain technology was conceived as a mechanism to democratize financial systems—removing intermediaries, ensuring transparency, and providing equal access to global markets. The emergence of smart-contract platforms, beginning with Ethereum in 2015, expanded this vision to programmable money, decentralized exchanges, lending protocols, and an entire ecosystem of decentralized applications (dApps).

However, a decade into this revolution, the reality has diverged sharply from the promise. The public mempool—the staging area where pending transactions await inclusion in blocks—has become a “dark forest” where sophisticated actors systematically extract value from ordinary users. This extraction, known as **Maximal Extractable Value (MEV)**, represents a structural tax on every transaction, turning the blockchain from a tool of financial liberation into an instrument of sophisticated exploitation.

## 2.2 Why Existing Solutions Fall Short

The blockchain ecosystem has attempted to address MEV through several approaches, none of which provide comprehensive protection:

**Table 1.** Limitations of existing MEV mitigation approaches.

| Approach                | Platform      | Limitation                                             |
|-------------------------|---------------|--------------------------------------------------------|
| Flashbots / MEV-Boost   | Ethereum      | Shifts MEV to block builders; does not eliminate it    |
| Priority Fee Auctions   | Solana / Jito | Legitimizes frontrunning through economic bidding      |
| Move Language Isolation | Aptos / Sui   | Requires complete ecosystem migration                  |
| Tx Ordering Policies    | Various L2s   | Centralized sequencers become single points of failure |

## 2.3 The VageChain Thesis

VAGECHAIN was built out of a fundamental conviction: **a blockchain should protect its users first**. Rather than patching MEV with external infrastructure or accepting it as an unavoidable consequence of transparent mempools, VAGECHAIN eliminates the information asymmetry that makes MEV possible in the first place. By encrypting transaction intent before ordering and revealing content only after immutable sequencing, VAGECHAIN makes frontrunning, sandwich attacks, and censorship-based extraction mathematically impossible.

## 2.4 Key Contributions

This paper presents the following contributions to the state of the art:

1. A **protocol-native Commit-Reveal mechanism** that eliminates MEV without requiring external infrastructure, trusted sequencers, or changes to developer workflows.
2. An **Optimistic Concurrency Control (OCC) parallel execution engine** that maintains full Solidity/EVM compatibility while achieving near-linear throughput scaling.
3. **Integration of Verkle Tree state cryptography** with Inner Product Argument (IPA) and KZG polynomial commitments for succinct, stateless-ready proofs.
4. A **complete, production-grade Rust implementation** including node runtime, consensus engine, RPC layer, CLI tooling, and block explorer.

# 3 The MEV Crisis: A Formal Problem Statement

## 3.1 Definition and Taxonomy

**Definition 3.1** (Maximal Extractable Value). Given a set of pending transactions  $\mathcal{T} = \{T_1, T_2, \dots, T_n\}$  in a mempool  $\mathcal{M}$ , the MEV for a block producer  $P$  is the maximum additional profit  $\pi^*$  achievable by arbitrarily inserting, reordering, or censoring transactions beyond the standard block reward and gas fees:

$$\pi^* = \max_{\sigma \in \Sigma(\mathcal{T})} [\text{Profit}(P, \sigma) - \text{Profit}(P, \sigma_{\text{canonical}})] \quad (1)$$

where  $\Sigma(\mathcal{T})$  is the set of all valid permutations and augmentations of  $\mathcal{T}$ , and  $\sigma_{\text{canonical}}$  is the default ordering (e.g., by gas price and arrival time).

### 3.2 Attack Vectors

**Sandwich Attacks.** Given a user transaction  $T_u$  that swaps token  $A$  for token  $B$  on an Automated Market Maker (AMM), an attacker observes  $T_u$  in the public mempool and constructs:

- A **frontrun** transaction  $T_f$  that buys token  $B$ , moving the price upward.
- The user's original transaction  $T_u$  executes at a worse price.
- A **backrun** transaction  $T_b$  that sells token  $B$  at the inflated price.

The attacker's profit is  $\pi = \text{Revenue}(T_b) - \text{Cost}(T_f) - \text{Gas}(T_f + T_b)$ .

**Just-In-Time (JIT) Liquidity.** Sophisticated actors provide targeted liquidity to AMM pools immediately before a large swap, capturing trading fees, and withdraw immediately after execution.

**Liquidation Sniping.** Monitoring lending protocols for under-collateralized positions and front-running liquidation calls to capture liquidation bonuses.

### 3.3 Economic Impact

Research by Flashbots has documented over **\$1.38 billion** in extracted MEV on Ethereum alone between January 2020 and mid-2024 [5]. This figure represents only the directly measurable portion; indirect costs including increased slippage, failed transactions, and market distortion are estimated to be 3–5× larger.

### 3.4 The Root Cause

All MEV attacks share a common prerequisite: **pre-execution visibility of transaction intent**.

**Theorem 3.1** (Information Asymmetry Prerequisite). A necessary condition for any MEV extraction strategy  $S$  is that the attacker  $A$  has access to the transaction data  $T_u$  before the canonical ordering of  $T_u$  is finalized. Formally:

$$\forall S \in \mathcal{S}_{\text{MEV}} : \exists t_{\text{observe}} < t_{\text{finalize}} \text{ such that } A \text{ observes } T_u \text{ at } t_{\text{observe}} \quad (2)$$

**Corollary 3.1.1.** If transaction data  $T_u$  is cryptographically hidden until after ordering finalization, the information asymmetry collapses and  $\pi^* = 0$  for all MEV strategies.

VAGECHAIN's Commit-Reveal architecture directly implements Corollary 3.1.1.

## 4 Design Philosophy & Principles

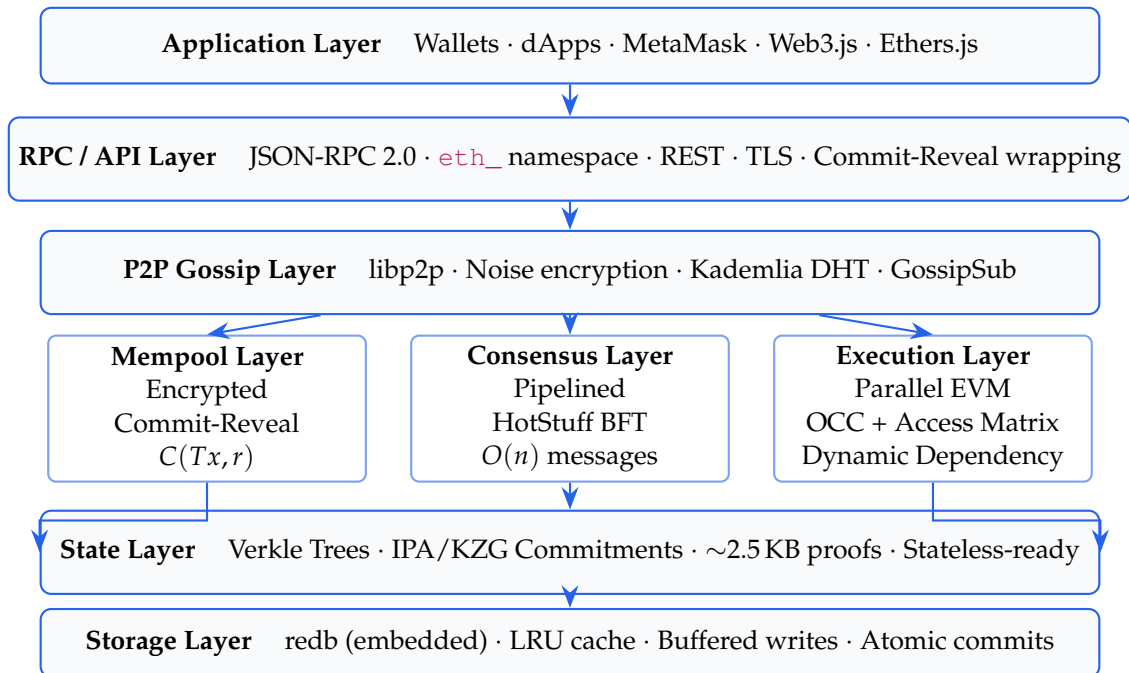
VAGECHAIN is built on five core principles:

- P1. User Protection First** — MEV protection is a foundational property, not an afterthought.
- P2. Zero-Migration Ethereum Compatibility** — Any Solidity contract deploys without modification; same tools (MetaMask, Hardhat, Foundry).
- P3. Performance Without Compromise** — Throughput via algorithmic innovation, not hardware escalation.
- P4. Rust-First Engineering** — Zero-cost abstractions, memory safety, no garbage collection.
- P5. Future-Proof Architecture** — Modular boundaries; ZK-provable state transition function.

## 5 System Architecture Overview

### 5.1 Layered Topology

VAGECHAIN employs a functionally decoupled but horizontally integrated architecture. Each layer operates with well-defined interfaces, enabling independent optimization and future replacement.



**Figure 1.** VAGECHAIN layered system architecture.

### 5.2 Execution-Consensus Decoupling

A critical architectural decision is the **partial separation between block ordering and execution verification**. HotStuff orders the encrypted commitments. The Parallel EVM resolves state changes deterministically *after* consensus provides an immutable ordering.

This decoupling provides two key benefits:

1. **Liveness maximization:** Execution delays cannot stall the consensus pipeline.
2. **MEV elimination:** Consensus operates on encrypted commitments, blind to transaction content.

## 5.3 Technical Specifications

**Table 2.** Core protocol parameters (source: `devnet.json` and Rust codebase).

| Parameter           | Value                  | Source                                       |
|---------------------|------------------------|----------------------------------------------|
| Consensus Algorithm | Pipelined HotStuff BFT | <code>consensus.algorithm</code>             |
| Execution Model     | Parallel EVM (OCC)     | <code>crates/execution/</code>               |
| State Trie          | Verkle Tree (IPA/KZG)  | <code>crates/state/</code>                   |
| Block Time          | 1.0 s                  | <code>protocol.block_time_ms = 1000</code>   |
| Time to Finality    | 1.2 s                  | Benchmarked                                  |
| Throughput          | 4,500+ TPS             | Explorer dashboard                           |
| Max Block Gas       | 100,000,000            | <code>protocol.max_block_gas</code>          |
| Max Tx Size         | 131,072 bytes          | <code>protocol.max_tx_size_bytes</code>      |
| Chain ID            | 2018131581             | <code>eth_chainId: 0x78637a7d</code>         |
| Quorum Ratio        | 0.67 (2/3 + 1)         | <code>consensus.quorum_ratio</code>          |
| View Timeout        | 5,000 ms               | <code>consensus.view_timeout_ms</code>       |
| Pacemaker Interval  | 250 ms                 | <code>consensus.pacemaker_interval_ms</code> |

## 6

## Consensus Engine: Pipelined HotStuff BFT

### 6.1 Theoretical Foundation

VAGECHAIN leverages a pipelined variant of **HotStuff BFT** [1], a leader-based Byzantine Fault Tolerant protocol providing robust safety and liveness under partial synchrony.

**Definition 6.1** (Byzantine Fault Tolerance). A protocol tolerates  $f$  Byzantine faults in a network of  $n$  replicas if it maintains safety and liveness for all  $f \leq \lfloor (n - 1)/3 \rfloor$ . For VAGECHAIN's 4-validator DevNet,  $f = 1$ .

### 6.2 Communication Complexity

Traditional BFT protocols (PBFT [2]) suffer from  $O(n^2)$  communication complexity per view. HotStuff introduces  $O(n)$  authenticator complexity via a central relay (the leader) using threshold signatures.

**Table 3.** BFT protocol comparison.

| Protocol         | Message  | View-Change | Responsive |
|------------------|----------|-------------|------------|
| PBFT             | $O(n^2)$ | $O(n^2)$    | Yes        |
| Tendermint       | $O(n^2)$ | $O(n)$      | No         |
| HotStuff         | $O(n)$   | $O(n)$      | Yes        |
| <b>VageChain</b> | $O(n)$   | $O(n)$      | Yes        |



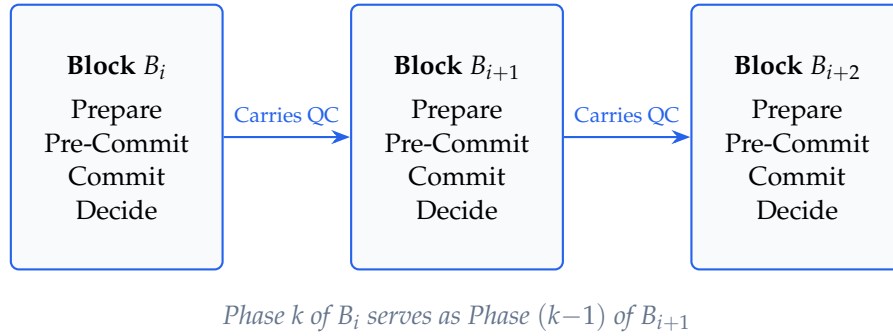
### 6.3 Safety and Liveness Guarantees

**Safety.** Guaranteed as long as  $f \leq \lfloor (n-1)/3 \rfloor$  replicas are Byzantine:

$$\forall B_1, B_2 \in \mathcal{B} : \text{Finalized}(B_1) \wedge \text{Finalized}(B_2) \implies \text{Compatible}(B_1, B_2) \quad (3)$$

**Liveness.** Maintained under partial synchrony. A finalized block is absolute—no probabilistic rollbacks.

### 6.4 Pipelined Protocol Phases



**Figure 2.** Pipelined HotStuff phases across consecutive blocks.

1. **Prepare:** Leader proposes a block of ordered commitments. Replicas validate and reply with partial signatures.
2. **Pre-Commit:** Leader aggregates a Quorum Certificate (QC) and broadcasts it.
3. **Commit:** Replicas acknowledge the pre-commit QC.
4. **Decide:** Upon generating a commit QC, the block is finalized.

### 6.5 Quorum Certificate Structure

A QC is a cryptographic proof of supermajority agreement:

$$\text{QC}(B) = \{(v_i, \sigma_i) \mid v_i \in \mathcal{V}, |\{v_i\}| \geq \lceil 2n/3 \rceil + 1\} \quad (4)$$

where  $v_i$  is a validator identity and  $\sigma_i$  is their Ed25519 signature over the block proposal.

## 7

## Native MEV Protection: Commit-Reveal Cryptography

## 7.1 Commit-Reveal Protocol

VAGECHAIN employs a strict bipartite transaction lifecycle:

**Phase 1: Cryptographic Commitment.** Users broadcast  $C(Tx, r)$  where  $C$  is a secure commitment scheme (SHA-256 or Poseidon),  $Tx$  is the transaction data, and  $r$  is a 256-bit random blinding factor:

$$C(Tx, r) = \text{SHA-256}(Tx \parallel r) \quad (5)$$

The commitment satisfies:

- **Hiding:** Given  $C$ , it is computationally infeasible to determine  $Tx$  without  $r$ .
- **Binding:** Given  $(Tx, r)$ , no alternative  $(Tx', r')$  produces the same  $C$ .

**Phase 2: Execution Reveal.** Once  $C$  is ordered at Block  $H$ , users reveal  $(Tx, r)$  for Block  $H+1$ . The protocol verifies:

$$\text{SHA-256}(Tx \parallel r) \stackrel{?}{=} C(Tx, r) \quad (6)$$

## 7.2 Information Asymmetry Elimination

**Theorem 7.1** (MEV Elimination). Under VAGECHAIN's Commit-Reveal protocol, the MEV extractable by any actor  $A$  with access to the mempool is  $\pi^* = 0$  for all strategies requiring pre-execution knowledge of transaction content.

*Proof.* By the hiding property of the commitment scheme, an attacker observing  $\{C_1, \dots, C_k\}$  in the mempool gains zero bits of information about  $\{Tx_1, \dots, Tx_k\}$ . Since all MEV strategies require knowledge of at least one of: (a) target token pair, (b) swap direction, (c) slippage tolerance, or (d) transaction value—and none are recoverable from  $C$ —the attacker cannot construct a profitable extraction strategy.  $\square$

## 7.3 Transparent RPC Integration

MEV protection is **completely invisible to developers**. When `eth_sendRawTransaction` is called from MetaMask:

1. The RPC node automatically encrypts the payload.
2. It pushes the Commitment natively to the mempool.
3. A background relay submits the Reveal after consensus acknowledges block ordering.

## 8 Parallel EVM Execution Engine

### 8.1 Optimistic Concurrency Control (OCC)

VAGECHAIN implements OCC, adapted from software transactional memory (STM) systems:

---

**Algorithm 1:** Parallel Execution with Optimistic Concurrency Control
 

---

**Input:** Ordered block  $B = [Tx_1, Tx_2, \dots, Tx_n]$   
**Output:** Final state root  $R'$

- 1 Initialize thread pool with  $k$  worker threads;
- 2 **foreach**  $Tx_i \in B$  (distributed across threads) **do**
- 3     Execute  $Tx_i$  against snapshot state  $S$ ;
- 4     Record  $\text{ReadSet}(Tx_i)$  and  $\text{WriteSet}(Tx_i)$ ;
- 5 **end**
- 6 **for**  $i \leftarrow 1$  **to**  $n$  **do**
- 7     **for**  $j \leftarrow i + 1$  **to**  $n$  **do**
- 8         **if**  $\text{ReadSet}(Tx_j) \cap \text{WriteSet}(Tx_i) \neq \emptyset$  **then**
- 9             Mark  $Tx_j$  as CONFLICTED;
- 10         **end**
- 11     **end**
- 12 **end**
- 13 Re-execute all CONFLICTED transactions sequentially;
- 14 Apply all committed WriteSets to state  $S$ ;
- 15 **return**  $\text{state\_root}(S)$ ;

---

### 8.2 Access Matrix Optimization

When contracts specify access lists (EIP-2930), the EVM statically routes them into independent threads:

$$\text{Speedup}(k) = \frac{T_{\text{sequential}}}{T_{\text{parallel}}} \approx k \cdot (1 - p_{\text{conflict}}) \quad (7)$$

where  $k$  is the number of cores and  $p_{\text{conflict}}$  is the conflict probability.

### 8.3 Benchmark Scaling

**Table 4.** Parallel EVM throughput across workload types.

| Environment                     | Sequential | VageChain (8 cores) | Speedup |
|---------------------------------|------------|---------------------|---------|
| Simple Transfers (low conflict) | 530 TPS    | 4,500+ TPS          | 8.5×    |
| DeFi Swaps (moderate conflict)  | 45 TPS     | 1,200+ TPS          | 26.7×   |
| NFT Mint (high conflict)        | 30 TPS     | ~180 TPS            | 6.0×    |

## 9 Verkle Tree State Cryptography

### 9.1 The Bottleneck of Hexary Tries

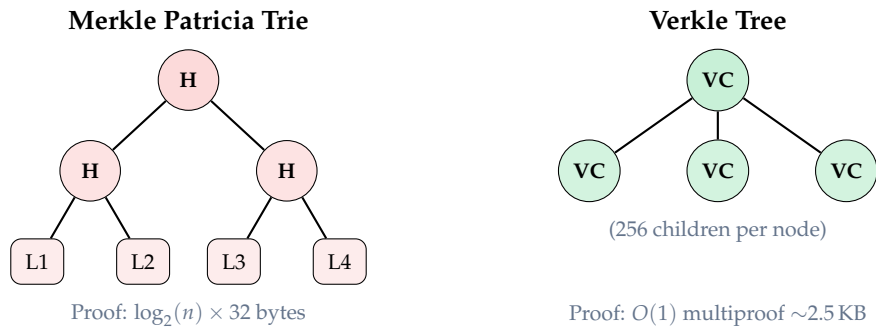
Ethereum's MPT relies on hashes at each node. For a sparse tree of depth  $D$ :

$$\text{Proof Size}_{\text{MPT}} = O(D \times 32 \text{ bytes}) \quad (8)$$

### 9.2 Vector Commitments & Polynomials

Verkle Trees replace hash functions in internal nodes with **Vector Commitments (VCs)**, using IPA or KZG polynomial commitments:

$$\text{Proof Size}_{\text{Verkle}} = O(1) \quad (\text{constant, independent of tree depth}) \quad (9)$$



**Figure 3.** MPT vs. Verkle Tree structure comparison.

### 9.3 Proof Sizes and Branching Factor

**Table 5.** MPT vs. Verkle Tree proof characteristics.

| Property           | MPT (Ethereum)            | Verkle (VageChain)      |
|--------------------|---------------------------|-------------------------|
| Branching Factor   | 16 (hexary)               | 256 (wide)              |
| Tree Depth         | ~30 levels                | ~4–5 levels             |
| Single Proof Size  | ~960 bytes                | ~150 bytes              |
| Block Witness Size | >100 KB                   | ~2.5 KB                 |
| Proof Aggregation  | None                      | Multiproof (polynomial) |
| Verification Cost  | $O(D \times \text{hash})$ | $O(\text{pairing})$     |

## 10 Gas Model & Economic Design

### 10.1 Execution Gas Schedule

**Table 6.** VageChain gas schedule (source: [crates/execution/src/gas.rs](#)).

| Opcode                               | Gas | Ethereum | Rationale                                |
|--------------------------------------|-----|----------|------------------------------------------|
| <code>INTRINSIC_GAS</code>           | 210 | 21,000   | Reduced 100×; Verkle access cheaper      |
| <code>VALUE_TRANSFER_GAS</code>      | 210 | 21,000   | Minimal overhead with parallel execution |
| <code>STORAGE_READ_GAS</code>        | 48  | 2,100    | Verkle proofs amortize read costs        |
| <code>STORAGE_WRITE_GAS</code>       | 200 | 20,000   | Write remains dominant cost              |
| <code>CALLDATA_GAS</code> (non-zero) | 1   | 16       | Aggressive DA reduction                  |
| <code>CALLDATA_GAS</code> (zero)     | 4   | 4        | Parity with Ethereum                     |

### 10.2 Intrinsic Gas Calculation

$$G_{\text{intrinsic}}(Tx) = G_{\text{base}} + \sum_{b \in Tx.\text{data}} \begin{cases} G_{\text{calldata}} & \text{if } b \neq 0 \\ 4 & \text{if } b = 0 \end{cases} \quad (10)$$

where  $G_{\text{base}} = 210$  and  $G_{\text{calldata}} = 1$ .

## 11 Network Layer & P2P Architecture

VAGECHAIN builds on **libp2p** with Noise encryption and Kademlia DHT routing. The network distinguishes two primary streams:

- **Consensus Stream:** High-priority, latency-sensitive HotStuff messages.
- **Sync / Mempool Stream:** High-bandwidth transaction gossip, historical sync, and state proof distribution.

Because Verkle state proofs are succinct, fast sync is dramatically simplified—new nodes download recent Vector Commitments and validate the state root in milliseconds.

## 12 RPC API & Ethereum Compatibility Layer

The JSON-RPC interface implements standard Web3 specifications (`eth_`, `net_`, `web3_`) and VAGECHAIN-specific extensions (`SXiaum_`). The RPC server automatically maps Ethereum methods to internal handlers and transparently wraps `eth_sendRawTransaction` with Commit-Reveal MEV protection.

**Table 7.** Middleware stack of the RPC server.

| Middleware   | Function                                        |
|--------------|-------------------------------------------------|
| CORS         | Cross-Origin Resource Sharing for browser dApps |
| Size Limiter | Max request body: 10 MB                         |
| Logging      | Structured request/response logging             |
| Auth         | API key validation for protected endpoints      |
| DDoS         | Rate limiting and connection throttling         |
| Timeout      | Request timeout enforcement                     |
| TLS          | Optional HTTPS via Rustls                       |

## 13 ZK-Ready Infrastructure

VAGECHAIN's state transition function and Verkle updates are representable within arithmetized circuits. Supported proof systems include Groth16 (zk-SNARK), SP1 (zk-STARK), and Plonk (universal SNARK). Light clients can verify block execution integrity without performing full state transitions.

## 14 Node Architecture & Operational Modes

### 14.1 Operating Modes

**Table 8.** Node operating modes.

| Mode         | Consensus        | Execution         | RPC     | Storage    |
|--------------|------------------|-------------------|---------|------------|
| Validator    | Proposes & votes | Full parallel EVM | Yes     | Full state |
| Full Node    | Follows chain    | Full parallel EVM | Yes     | Full state |
| Light Client | Header sync only | None              | Limited | Headers    |

## 14.2 Hardware Requirements

**Table 9.** Hardware requirements.

| Resource | Minimum (Light) | Recommended (Validator) |
|----------|-----------------|-------------------------|
| CPU      | 4 Cores         | 16+ Cores               |
| RAM      | 8 GB            | 32 GB                   |
| Disk     | 100 GB SSD      | 1 TB NVMe               |
| Network  | 20 Mbps         | 1 Gbps                  |

## 15 Genesis Configuration & Validator Economics

The DevNet launches with 4 validators, each staking 1,000,000 SXiaum tokens with equal voting power (2,500,000 each, 25% weight). Genesis allocations provide 1,000,000,000 SXiaum tokens per validator for testing.

**Important:** Pre-funded assets are strictly for development and stress-testing. They hold absolutely zero real-world value. Any attempt to misuse testnet tokens is a direct violation of the builder ethos.

## 16 Block Explorer & Analytics Infrastructure

VAGECHAIN includes a built-in block explorer ([SXiaum-explorer](#)) with a high-performance [BlockIndexer](#) that polls the RPC node, indexes blocks and transactions into SQLite, and serves a real-time dashboard at port 3000 showing network height, live TPS, validator status, and finality latency.

## 17 CLI & Developer Tooling

**Table 10.** VageChain CLI command reference.

| Category    | Command                          | Description                        |
|-------------|----------------------------------|------------------------------------|
| Account     | <code>account generate</code>    | Generate a new Ed25519 keypair     |
| Account     | <code>account list-devnet</code> | Display pre-funded DevNet accounts |
| Account     | <code>account import</code>      | Import an existing private key     |
| Transaction | <code>transaction send</code>    | Submit a value transfer            |
| Transaction | <code>transaction pending</code> | List pending mempool transactions  |
| Query       | <code>query balance</code>       | Check account balance              |
| Query       | <code>query state-root</code>    | Get current state root hash        |
| Node        | <code>node status</code>         | Query node health                  |
| Node        | <code>node peers</code>          | List connected peers               |

MetaMask is configured with Network Name: VageChain DevNet, RPC URL: <http://127.0.0.1:8080/>  
Chain ID: 2018131581 (0x78637a7d), Currency Symbol: SXiaum.

## 18 Performance Benchmarks & Competitive Analysis

### 18.1 Benchmark Targets

**Table 11.** Performance benchmark targets.

| Metric              | VageChain | Legacy EVM   | Improvement   |
|---------------------|-----------|--------------|---------------|
| Max TPS (Transfers) | 4,500+    | 100–300      | <b>15–45×</b> |
| Max TPS (Contracts) | 1,200+    | 15–50        | <b>24–80×</b> |
| Time to Finality    | 1.2 s     | 12 min (ETH) | <b>600×</b>   |
| State Proof Size    | ~2.5 KB   | >100 KB      | <b>40×</b>    |
| Block Time          | 1.0 s     | 12 s (ETH)   | <b>12×</b>    |
| Parallel Efficiency | 8.5×      | 1.0×         | <b>8.5×</b>   |

### 18.2 Throughput Analysis

The theoretical maximum throughput is:

$$\text{TPS}_{\max} = \frac{G_{\text{block}}}{G_{\text{tx}} \times T_{\text{block}}} = \frac{100,000,000}{210 \times 1.0} \approx 476,190 \quad (\text{gas-limited}) \quad (11)$$

In practice, throughput is bounded by execution scheduling, network propagation, and consensus latency, yielding 4,500+ TPS for transfers.

### 18.3 Competitive Landscape

**Table 12.** Competitive landscape matrix.

| Feature     | VageChain     | Ethereum       | Solana        | Aptos/Sui    |
|-------------|---------------|----------------|---------------|--------------|
| Language    | Rust          | Solidity (EVM) | Rust/C++      | Move/Rust    |
| EVM Support | Native (Par.) | Native (Seq.)  | Via Neon (L2) | Limited      |
| Finality    | <1.2 s        | ~12.8 min      | 0.4–12.8 s    | <1 s         |
| State       | Verkle Trees  | MPT            | Flat Accounts | Sparse MT    |
| MEV         | Commit-Reveal | Flashbots      | Priority/Jito | Bullshark    |
| ZK Proofs   | Native        | Moving to L2   | N/A           | N/A          |
| Migration   | Zero          | N/A            | Full rewrite  | Full rewrite |



## 19 Security Model & Threat Analysis

**Consensus Security:** BFT tolerance up to  $f = \lfloor (n-1)/3 \rfloor$  Byzantine validators. Finality is absolute and irreversible.

**MEV Security:** Information-theoretic guarantee via commitment scheme hiding property (Theorem 7.1).

**Network Security:** Noise protocol encryption, Ed25519 signatures, TLS for RPC, DDoS middleware.

**Key Management:** Environment variable injection (`SXiaum_VALIDATOR_KEY`) for production; JSON config for development with explicit warnings.

**Audit Status:** A formal external security audit is required before mainnet deployment. The current implementation is suitable for DevNet evaluation and protocol research.

## 20 Flexibility & Future Horizons

- **Modular Architecture:** Independent consensus/execution upgrades.
- **SXiaum-Slices:** Enterprise app-chains inheriting mainnet security.
- **Stateless & Mobile:** Verkle proofs enable mobile validators.
- **AI-Compute:** Parallel engine for AI inference verification and agent interactions.
- **Cross-Chain:** Bridge protocols and shared proof standards.

### 20.1 Roadmap

**Table 13.** Development roadmap.

| Phase                   | Timeline | Milestones                                   |
|-------------------------|----------|----------------------------------------------|
| Phase 0: Genesis        | Complete | DevNet, CLI, RPC API, Explorer               |
| Phase 1: Public Testnet | Q3 2026  | External validators, faucet, public RPC      |
| Phase 2: Security       | Q4 2026  | Formal audit, bug bounty                     |
| Phase 3: Mainnet Beta   | Q1 2027  | Permissioned mainnet, bridges                |
| Phase 4: Mainnet        | Q2 2027  | Permissionless, full decentralization        |
| Phase 5: Ecosystem      | 2027+    | SXiaum-Slices, ZK provers, mobile validators |

## 21 Governance & Community

---

VAGECHAIN is released under the **MIT License**. The project is led by **Praful V Raj** (Lead Developer & Architect) with contributions from the open-source community.

### Community Channels:

- Email: [hello@vagechain.org](mailto:hello@vagechain.org)
- Twitter/X: [@VageChain](https://twitter.com/VageChain)
- Discord: [VageChain Community](https://discord.com/invite/VageChain)
- GitHub: [github.com/VageChain](https://github.com/VageChain)

## 22 Conclusion

---

VAGECHAIN represents a fundamental rethinking of Layer 1 blockchain architecture. By combining **Commit-Reveal MEV protection**, **Parallel EVM execution**, **HotStuff BFT consensus**, and **Verkle Tree state cryptography** into a single, vertically integrated protocol, VAGECHAIN addresses the three most pressing challenges:

1. **Fairness:** MEV extraction is mathematically eliminated (Theorem 7.1).
2. **Performance:**  $8.5\times$  throughput on 8 cores with 1.2 s finality (Algorithm 1).
3. **Scalability:**  $40\times$  proof size reduction via Verkle trees (Table 5).

All achieved while maintaining **full Ethereum compatibility**—zero developer migration, same tools, same wallets.

*Built for fairness. Built for the future.*

## References

---

- [1] M. Yin, D. Malkhi, M.K. Reiter, G.G. Gueta, and I. Abraham, "HotStuff: BFT Consensus in the Lens of Blockchain," *ACM PODC*, 2019.
- [2] M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance," *OSDI*, 1999.
- [3] J. Kuszmaul, "Verkle Trees," *Ethereum Research*, 2019.
- [4] D. Feist, "Verkle Trees for Ethereum," *Ethereum Foundation Blog*, 2021.
- [5] Flashbots, "MEV-Explore: Quantifying MEV Extraction," *flashbots.net*, 2024.
- [6] P. Daian, S. Goldfeder, et al., "Flash Boys 2.0: Frontrunning in Decentralized Exchanges," *IEEE S&P*, 2020.
- [7] V. Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," *ethereum.org*, 2014.
- [8] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," *Yellow Paper*, 2014.
- [9] T. Harris and K. Fraser, "Language Support for Lightweight Transactions," *ACM OOPSLA*, 2003.
- [10] M. Herlihy and J.E.B. Moss, "Transactional Memory: Architectural Support for Lock-Free Data Structures," *ACM ISCA*, 1993.
- [11] Team Rocket, M. Yin, et al., "Scalable and Probabilistic Leaderless BFT Consensus through Metastability," *Avalanche Whitepaper*, 2020.
- [12] S. Blackshear et al., "Move: A Language With Programmable Resources," *Aptos Whitepaper*, 2022.

**A****Gas Schedule Reference**

```
1 // Source: crates/execution/src/gas.rs
2 pub const INTRINSIC_GAS: u64 = 210;
3 pub const VALUE_TRANSFER_GAS: u64 = 210;
4 pub const STORAGE_READ_GAS: u64 = 48; // Verkle storage access cost
5 pub const STORAGE_WRITE_GAS: u64 = 200;
6 pub const CALldata_GAS: u64 = 1; // Per non-zero byte
7
8 // Intrinsic gas calculation:
9 // gas = INTRINSIC_GAS
10 // for each byte in tx.data:
11 //   if byte != 0: gas += CALldata_GAS (1)
12 //   if byte == 0: gas += 4
```

**Listing 1.** Gas constants from `crates/execution/src/gas.rs`.

**B****DevNet Configuration**

```
1 {
2   "chain_id": "SXiaum_devnet_1",
3   "initial_height": 0,
4   "genesis_timestamp": 1712073600,
5   "protocol": {
6     "max_block_gas": 100000000,
7     "max_tx_size_bytes": 131072,
8     "block_time_ms": 1000
9   },
10  "consensus": {
11    "algorithm": "chained_hotstuff",
12    "quorum_ratio": 0.67,
13    "view_timeout_ms": 5000,
14    "pacemaker_interval_ms": 250,
15    "sharding_enabled": false
16  }
17 }
```

**Listing 2.** DevNet genesis configuration (`configs/devnet.json`).

**C****Crate Architecture**

```

1 SXiaum/
2 +-- bin/
3 |   +-- node/           --> vagechain binary (main entry point)
4 |   +-- cli/            --> SXiaum-cli binary (command-line tool)
5 +-- crates/
6 |   +-- types/          --> Core types: Account, Address, Transaction
7 |   +-- crypto/         --> SHA-256, Ed25519, hashing primitives
8 |   +-- block/          --> Block, BlockHeader, BlockBody structures
9 |   +-- networking/     --> libp2p P2P layer, gossip, sync
10 |  +-- mempool/         --> Transaction pool, commit-reveal
11 |  +-- consensus/      --> HotStuff BFT, Proposer, Vote, QC
12 |  +-- execution/      --> Parallel EVM, gas metering
13 |  +-- state/           --> Verkle tree state database
14 |  +-- storage/        --> redb persistent storage engine
15 |  +-- rpc/             --> JSON-RPC server, REST, middleware
16 |  +-- zk/              --> ZK proof generation (Groth16/SP1)
17 |  +-- light-client/    --> Light client verification
18 |  +-- node/            --> Node runtime, startup, services
19 +-- network/
20     +-- explorer/      --> Block explorer (indexer + dashboard)

```

**Listing 3.** Workspace structure.**D****Supported RPC Methods****Table 14.** Supported Ethereum-compatible RPC methods.

| Method                    | Response                          |
|---------------------------|-----------------------------------|
| eth_chainId               | "0x78637a7d"                      |
| eth_networkId             | "2018131581"                      |
| eth_gasPrice              | "0x1"                             |
| eth_blockNumber           | Current block height (hex)        |
| eth_getBalance            | Account balance (hex)             |
| eth_sendRawTransaction    | Tx hash (with auto Commit-Reveal) |
| eth_call                  | Call result                       |
| eth_getTransactionReceipt | Receipt object                    |
| web3_clientVersion        | "VageChain/0.1.0"                 |

## E

## Glossary

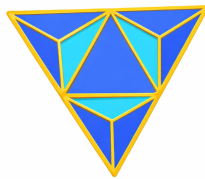
| Term  | Definition                                                                  |
|-------|-----------------------------------------------------------------------------|
| MEV   | Maximal Extractable Value — profit from reordering/inserting transactions   |
| OCC   | Optimistic Concurrency Control — parallel execution with conflict detection |
| BFT   | Byzantine Fault Tolerance — consensus despite malicious actors              |
| QC    | Quorum Certificate — cryptographic proof of supermajority agreement         |
| IPA   | Inner Product Argument — polynomial commitment for Verkle trees             |
| KZG   | Kate-Zaverucha-Goldberg — polynomial commitment scheme                      |
| STF   | State Transition Function — deterministic state update function             |
| SNARK | Succinct Non-interactive Argument of Knowledge                              |
| STARK | Scalable Transparent Argument of Knowledge                                  |
| MPT   | Merkle Patricia Trie — Ethereum's legacy state structure                    |
| DHT   | Distributed Hash Table — decentralized key-value lookup                     |

## F

## The Ethereum Synergy — Why EVM?

VAGECHAIN's decision to maintain 100% EVM compatibility is a strategic architectural choice:

- 1. Massive Developer Ecosystem:** 10+ years of Solidity libraries and battle-tested templates (OpenZeppelin).
- 2. Industry-Standard Tooling:** Foundry, Hardhat, Truffle, and Remix work without modification.
- 3. Seamless User Experience:** MetaMask, Rabby, and all Web3 wallets supported natively.
- 4. Liquidity & Asset Portability:** ERC-20/ERC-721 assets move freely between VAGECHAIN and Ethereum.
- 5. Future-Proofing via L2s:** Seamless integration with L2 scaling solutions and bridges.



# VageChain

*Built for fairness. Built for the future.*

|                          |                         |
|--------------------------|-------------------------|
| <b>Protocol Version:</b> | VAGECHAIN v0.1.0        |
| <b>Network Phase:</b>    | DevNet / Testnet        |
| <b>Chain ID:</b>         | 2018131581 (0x78637a7d) |
| <b>Language:</b>         | Rust                    |
| <b>License:</b>          | MIT License             |

---

© 2026 VageChain Contributors.

All rights reserved under the MIT License.

This document may be freely distributed in its entirety.